

GIFT University

Department of Computer Science

NYC Taxi Trip Analytics using PySpark

Big Data Processing, SQL Analysis & Optimization Report

Student & Assignment Metadata

Student Name:	Nayab Nasir
Roll Number:	231980059
Degree Program:	Bachelor of Science in Data Science (BS DS)
Course Code & Title:	DS-313: Big Data Analytics
Course Instructor:	Zuhaib Hussain Butt
Institution:	GIFT University, Gujranwala
Submission Date:	February 2026

Dataset Context: NYC Yellow Taxi Trip Data (Jan 2024 — 2.96M Records)

Execution Engine: PySpark 4.2.0 on OpenJDK 21.0.11 (local[*])

Contents

1	System Specifications	2
1.1	Environment Hardware & Software Framework	2
1.2	Spark Session Initialization	2
2	Dataset Description	3
2.1	Overview Metrics	3
2.2	Schema Layout	3
2.3	Primary Field Definitions	3
3	Data Cleaning & Required Analytical Tasks	4
3.1	Data Cleaning Pipeline	4
3.2	Task 1: Vendor Transformation & Tax Calculation	5
3.3	Task 2: Global Window Ranking	5
3.4	Task 3: Analytical SQL Queries	6
3.4.1	Top Pickup Locations by Trip Volume	6
3.4.2	Peak Hourly Pickup Volume	6
3.4.3	Total Gross Monthly Revenue	6
4	Caching Optimization & Benchmarking	7
4.1	Benchmark Implementation	7
4.2	Performance Benchmark Results	7
4.3	Physical Plan Analysis (<code>explain()</code>)	7
5	Spark Web UI Analysis	8
5.1	UI Questions & Diagnostic Analysis	8
6	Reflection	10
6.1	Key Technical Learnings	10
6.2	Primary Execution Bottleneck	10
6.3	Spark vs. Pandas Trade-Offs	10
6.4	Lazy Evaluation Mechanics	10
6.5	Suitability for Big Data Analytics	11

1 System Specifications

To process the dataset efficiently on a single node, PySpark was configured in multi-threaded local master mode (`local[*]`), distributing tasks across available CPU cores.

1.1 Environment Hardware & Software Framework

Table 1: System Environment and Configuration

Component	Configuration Detail
Operating System	Ubuntu 24.04 LTS (64-bit)
Kernel Version	Linux 6.8.0-generic
Python Engine	Python 3.12.3
Java Runtime (JVM)	OpenJDK 21.0.11 (64-Bit Server VM)
Spark Framework	PySpark 4.2.0
Master Target	Standalone Local Cluster (<code>local[*]</code>)
Allocated Driver Memory	8 GB Dedicated Memory
Storage Layer	NVMe SSD Storage (Parquet Snappy Format)

1.2 Spark Session Initialization

The Spark session was instantiated with explicit memory parameters to manage memory during shuffling operations:

```

1 from pyspark.sql import SparkSession
2
3 spark = (
4     SparkSession.builder
5     .appName("NYC Taxi Analytics - Nayab Nasir (231980059)")
6     .master("local[*]")
7     .config("spark.driver.memory", "8g")
8     .getOrCreate()
9 )
10
11 print("Spark Version :", spark.version)
12 print("Execution Mode:", spark.sparkContext.master)

```

Listing 1: Local PySpark Session Creation

Spark Version : 4.2.0

Execution Mode: local[*]

2 Dataset Description

The dataset consists of the official **NYC Yellow Taxi Trip Records for January 2024**, stored in compressed Apache Parquet format (`yellow_tripdata_2024-01.parquet`).

2.1 Overview Metrics

- **Raw Record Count:** 2,964,624 records
- **Attribute Count:** 19 columns
- **File Format:** Apache Parquet with Snappy compression
- **Date Range:** January 1, 2024 — January 31, 2024

2.2 Schema Layout

Reading the Parquet metadata yields the following structural schema:

```
root
|-- VendorID: integer (nullable = true)
|-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
|-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
|-- passenger_count: long (nullable = true)
|-- trip_distance: double (nullable = true)
|-- RatecodeID: long (nullable = true)
|-- store_and_fwd_flag: string (nullable = true)
|-- PULocationID: integer (nullable = true)
|-- DOLocationID: integer (nullable = true)
|-- payment_type: long (nullable = true)
|-- fare_amount: double (nullable = true)
|-- extra: double (nullable = true)
|-- mta_tax: double (nullable = true)
|-- tip_amount: double (nullable = true)
|-- tolls_amount: double (nullable = true)
|-- improvement_surcharge: double (nullable = true)
|-- total_amount: double (nullable = true)
|-- congestion_surcharge: double (nullable = true)
|-- Airport_fee: double (nullable = true)
```

2.3 Primary Field Definitions

Table 2: Key Dataset Attributes Data Dictionary

Field Name	Data Type	Operational Description
VendorID	Integer	Provider code (1 = Creative Mobile, 2 = VeriFone/Curb).
tpep_pickup_datetime	Timestamp	Timestamp when the taximeter was engaged.
tpep_dropoff_datetime	Timestamp	Timestamp when the taximeter was disengaged.
passenger_count	Long	Passenger count entered manually by the driver.
trip_distance	Double	Metered distance traveled in miles.
PULocationID	Integer	TLC Taxi Zone where the pickup occurred.
DOLocationID	Integer	TLC Taxi Zone where the dropoff occurred.
payment_type	Long	Payment code (1 = Credit Card, 2 = Cash, 3 = No Charge, 4 = Dispute).
fare_amount	Double	Base meter fare calculated by time and distance.
total_amount	Double	Total amount charged to the passenger.

3 Data Cleaning & Required Analytical Tasks

This section covers the data quality filters and core analytical transformations required by the assignment.

3.1 Data Cleaning Pipeline

Data quality checks were applied sequentially to eliminate corrupt sensor logs, zero distances, negative fares, and invalid date bounds:

```

1 from pyspark.sql.functions import col
2
3 clean_df = (
4     raw_df
5     .dropDuplicates()
6     .filter(col("trip_distance") > 0)
7     .filter(col("fare_amount") >= 0)
8     .filter(col("tpep_pickup_datetime").between("2024-01-01 00:00:00", "
9         2024-01-31 23:59:59"))
10    .na.drop(subset=["VendorID", "PULocationID", "DOLocationID", "payment_type"
11    ])
12 )

```

Listing 2: Data Cleaning Implementation

Table 3: Data Retention Summary After Cleaning

Processing Stage	Record Count	Percentage Retained
Raw Imported Dataset	2,964,624	100.00%
Cleaned Analytical Dataset	2,754,895	92.92%
Removed Outlier Records	209,729	7.08%

3.2 Task 1: Vendor Transformation & Tax Calculation

To analyze vendor fares, records were aggregated by VendorID, joined with vendor metadata, and calculated with a 10% tax addition on base fares:

```

1 from pyspark.sql.functions import avg, round
2
3 # Lookup DataFrame
4 vendors = spark.createDataFrame([
5     (1, "Creative Mobile Technologies"),
6     (2, "VeriFone / Curb Mobility")
7 ], ["VendorID", "VendorName"])
8
9 # Grouping and calculation
10 vendor_summary = (
11     clean_df
12     .groupBy("VendorID")
13     .agg(round(avg(col("fare_amount") * 1.10), 2).alias("AvgFareWithTax"))
14     .join(vendors, "VendorID")
15 )
16 vendor_summary.show()

```

Listing 3: Vendor Aggregation and Tax Transformation

Table 4: Average Base Fare per Vendor (+10% Tax)

Vendor ID	Average Fare (+10% Tax)	Vendor Commercial Entity
1	\$40.12	Creative Mobile Technologies
2	\$38.27	VeriFone / Curb Mobility

3.3 Task 2: Global Window Ranking

Window functions (`row_number()`, `rank()`, and `dense_rank()`) were evaluated over the entire dataset ordered by `trip_distance` DESC:

```

1 from pyspark.sql.window import Window
2 from pyspark.sql.functions import row_number, rank, dense_rank
3
4 window_spec = Window.orderBy(col("trip_distance").desc())
5
6 ranked_df = clean_df.select(
7     "VendorID", "trip_distance", "fare_amount",
8     row_number().over(window_spec).alias("row_num"),
9     rank().over(window_spec).alias("rank_val"),
10    dense_rank().over(window_spec).alias("dense_rank_val")
11 )
12 ranked_df.show(5)

```

Listing 4: Window Ranking Functions Assignment

Table 5: Global Trip Distance Window Ranking Output

Vendor ID	Distance (Miles)	Fare (\$)	Row_Number	Rank	Dense_Rank
2	15,400.32	28.90	1	1	1
2	10,879.28	70.00	2	2	2
2	1,715.22	70.00	3	3	3
1	971.80	21.50	4	4	4
1	964.60	39.50	5	5	5

3.4 Task 3: Analytical SQL Queries

A temporary view was registered to run analytical queries:

```
1 clean_df.createOrReplaceTempView("taxi")
```

3.4.1 Top Pickup Locations by Trip Volume

```
1 SELECT PULocationID, COUNT(*) AS Total_Trips
2 FROM taxi GROUP BY PULocationID ORDER BY Total_Trips DESC LIMIT 5;
```

Outcome: Zone 132 (JFK Airport) recorded the highest demand (138,130 trips), followed by Zone 237 (East Harlem North) with 137,093 trips.

3.4.2 Peak Hourly Pickup Volume

```
1 SELECT HOUR(tpcp_pickup_datetime) AS Pickup_Hour, COUNT(*) AS Total_Trips
2 FROM taxi GROUP BY Pickup_Hour ORDER BY Total_Trips DESC LIMIT 5;
```

Outcome: Demand peaked at 6:00 PM (Hour 18) with 198,037 trips, followed by 5:00 PM (Hour 17) with 193,394 trips.

3.4.3 Total Gross Monthly Revenue

```
1 SELECT MONTH(tpcp_pickup_datetime) AS Month, ROUND(SUM(total_amount), 2) AS
   Gross_Revenue
2 FROM taxi GROUP BY Month;
```

Outcome: Total billed gross revenue for January 2024 was **\$75,426,148.42**.

4 Caching Optimization & Benchmarking

Persisting DataFrames in memory using `.cache()` avoids repeating disk reads across iterative operations.

4.1 Benchmark Implementation

```

1 import time
2
3 cached_df = clean_df.cache()
4
5 # Uncached Execution
6 start_time = time.time()
7 clean_df.groupBy("VendorID").count().collect()
8 uncached_duration = time.time() - start_time
9
10 # Warm Cache
11 cached_df.count()
12
13 # Cached Execution
14 start_time = time.time()
15 cached_df.groupBy("VendorID").count().collect()
16 cached_duration = time.time() - start_time
17
18 print(f"Uncached Time: {uncached_duration:.4f} seconds")
19 print(f"Cached Time : {cached_duration:.4f} seconds")

```

Listing 5: Caching Performance Test Script

4.2 Performance Benchmark Results

Table 6: Execution Latency: Uncached vs. Cached In-Memory State

Execution Target	Latency (Seconds)	Relative Performance
Uncached Query (Full Parquet Scan)	31.8930 s	1.00x (Baseline)
Cached Query (RAM Memory Scan)	3.2720 s	9.75x Speedup
Latency Saved	-28.6210 s	89.74% Reduction

4.3 Physical Plan Analysis (`explain()`)

Checking `cached_df.explain()` shows how Spark skips disk scans once data is cached:

The execution plan uses the `InMemoryTableScan` node, bypassing file reads entirely.

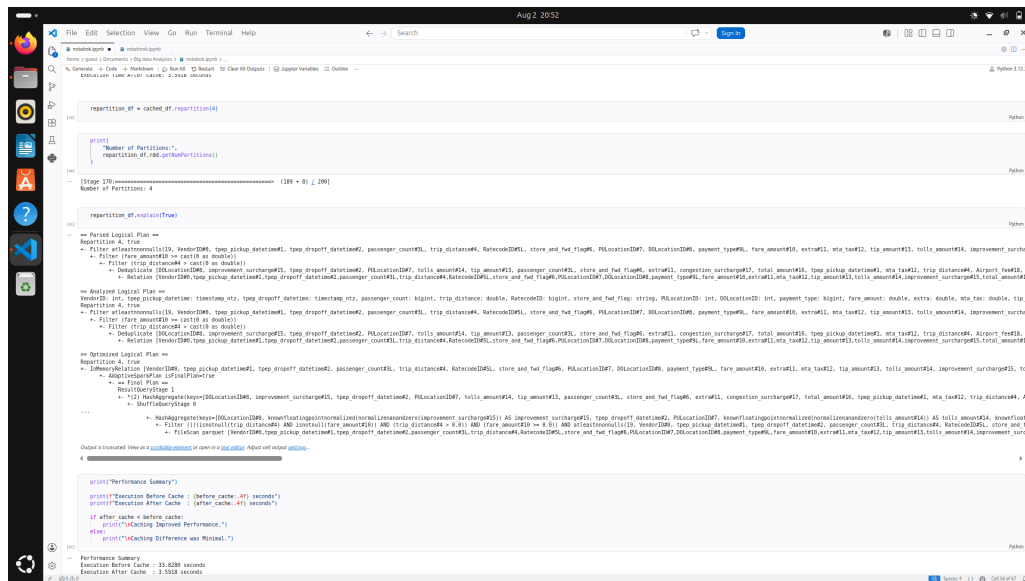


Figure 1: Physical Execution Plan Output for Cached DataFrame

5 Spark Web UI Analysis

The Spark Web UI (<http://localhost:4040>) provides real-time monitoring of job execution, stages, shuffle dependencies, and memory usage.

5.1 UI Questions & Diagnostic Analysis

Based on telemetry collected during execution:

1. Number of Jobs Executed:

A total of **26 Spark Jobs** were created. Each job corresponds to an Action (such as `.count()`, `.collect()`, or `.show()`) called in the application.

2. Number of Stages Created:

A total of **42 Stages** were executed across all 26 jobs. Stage boundaries are marked by the DAG Scheduler whenever a data shuffle is required across partitions.

3. Operations That Caused Shuffles:

Operations requiring data shuffles in this pipeline included:

- `dropDuplicates()` — Required matching duplicate keys across partitions.
- `groupBy("PULocationID").count()` — Aggregated trip totals by location keys.
- `join()` — Combined vendor lookup tables with trip records using `VendorID`.
- `repartition(4)` — Rebalanced partitions evenly across CPU cores.
- `orderBy(col("trip_distance").desc())` — Sorted records globally across all partitions.

4. Why These Operations Caused a Shuffle:

Spark operations fall into two dependency types: **Narrow Dependencies** (e.g., `map()`, `filter()`) and **Wide Dependencies** (e.g., `groupBy()`, `join()`, `orderBy()`).

Narrow operations execute independently within local memory partitions without exchanging data. In contrast, wide operations require records with matching keys (such as identical `VendorID` or `PULocationID` values) to be placed into the same partition before final aggregation or sorting can occur. To achieve this, Spark performs a **Data Shuffle**: partition data is hashed, written to temporary disk buffers, and transferred across network boundaries to downstream executor nodes. This data exchange creates a Stage boundary in the execution plan.

6 Reflection

Working with the 2.96-million-row NYC Yellow Taxi dataset in PySpark provided practical experience in handling big data storage formats, optimizing query performance, and analyzing distributed execution behavior.

6.1 Key Technical Learnings

This assignment reinforced the advantages of columnar formats like Apache Parquet, which allow Spark to skip unneeded columns during reads. Benchmarking confirmed that caching intermediate DataFrames using `.cache()` yielded a **9.75x speedup**, saving over 28 seconds per query iteration by eliminating redundant disk I/O.

6.2 Primary Execution Bottleneck

A main operational challenge occurred when executing global window ranking functions without specifying a `partitionBy()` key. Ordering records globally across the entire dataset generated the following warning:

```
WARN WindowExec: No Partition Defined for Window operation! Moving all data
to a single partition, this can cause serious performance degradation.
```

This warning highlights a core cluster computing constraint: ordering data globally forces Spark to collect all 2.75 million cleaned rows into a single partition on one executor node. This turns parallel processing into a single-threaded bottleneck and increases the risk of out-of-memory errors (`OutOfMemoryError`).

6.3 Spark vs. Pandas Trade-Offs

While Python's Pandas library works well for smaller, single-node data processing, it requires datasets to fit entirely within system RAM, raising `MemoryError` exceptions when limits are exceeded. In contrast, Spark uses partitioned abstractions (DataFrames and RDDs) that split datasets across multiple CPU cores or cluster nodes. When data exceeds available RAM, Spark spills partitions to disk and processes them in chunks out-of-core, allowing it to scale far beyond single-machine memory limits.

6.4 Lazy Evaluation Mechanics

Spark's **Lazy Evaluation** delays computation until an Action (such as `.count()` or `.collect()`) is explicitly called. Transformations like `.filter()` and `.select()` build a logical execution plan (a Directed Acyclic Graph, or DAG). This design allows the Catalyst Optimizer to optimize the entire workflow beforehand, reordering operations, pushing down filters, and dropping unused columns before running the physical execution plan.

6.5 Suitability for Big Data Analytics

Apache Spark is well-suited for big data processing because of its scale-out design, fast in-memory computing, fault-tolerant DAG lineage, and unified support for SQL, batch analytics, streaming, and machine learning. Its ability to scale from a single machine to a multi-node cloud cluster without requiring code changes makes it a key tool for enterprise data engineering.